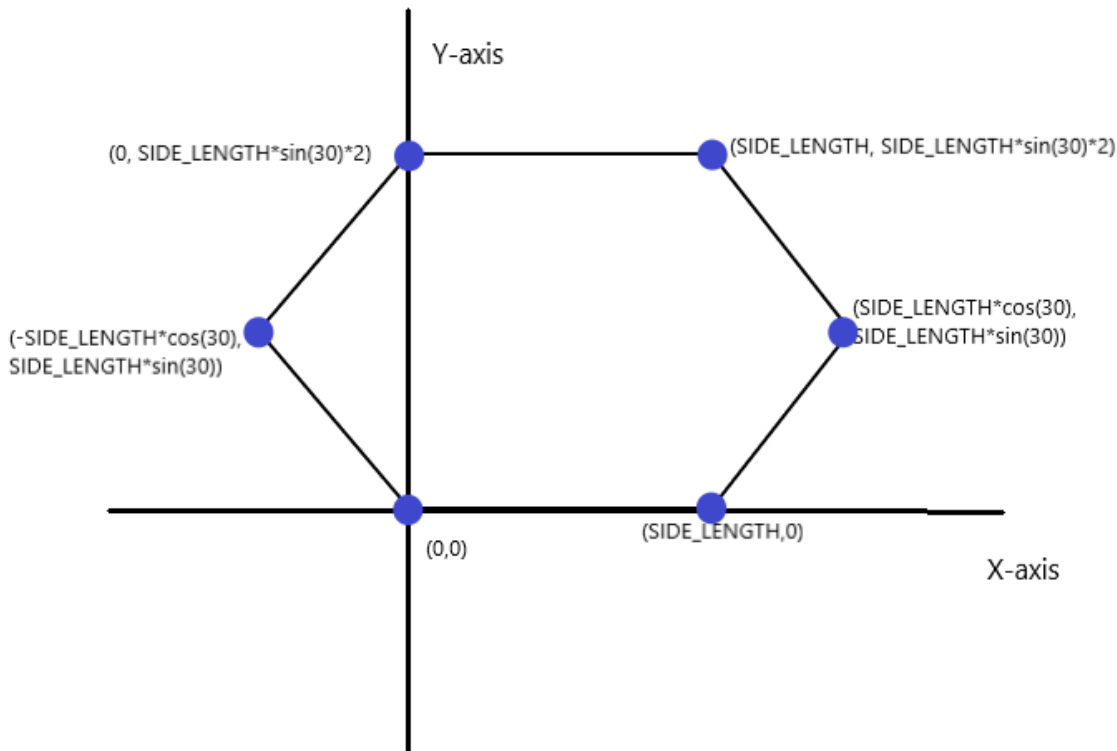


HEXAGON MISSION

1. In the function, `get_hexagon_points()`, saving hexagonal vertices in cartesian coordinates in `coords` list. Taking the starting point as origin.



2. Now converting those cartesian coordinates `(x, y)` into `(latitude, longitude)` by using the below transformation.

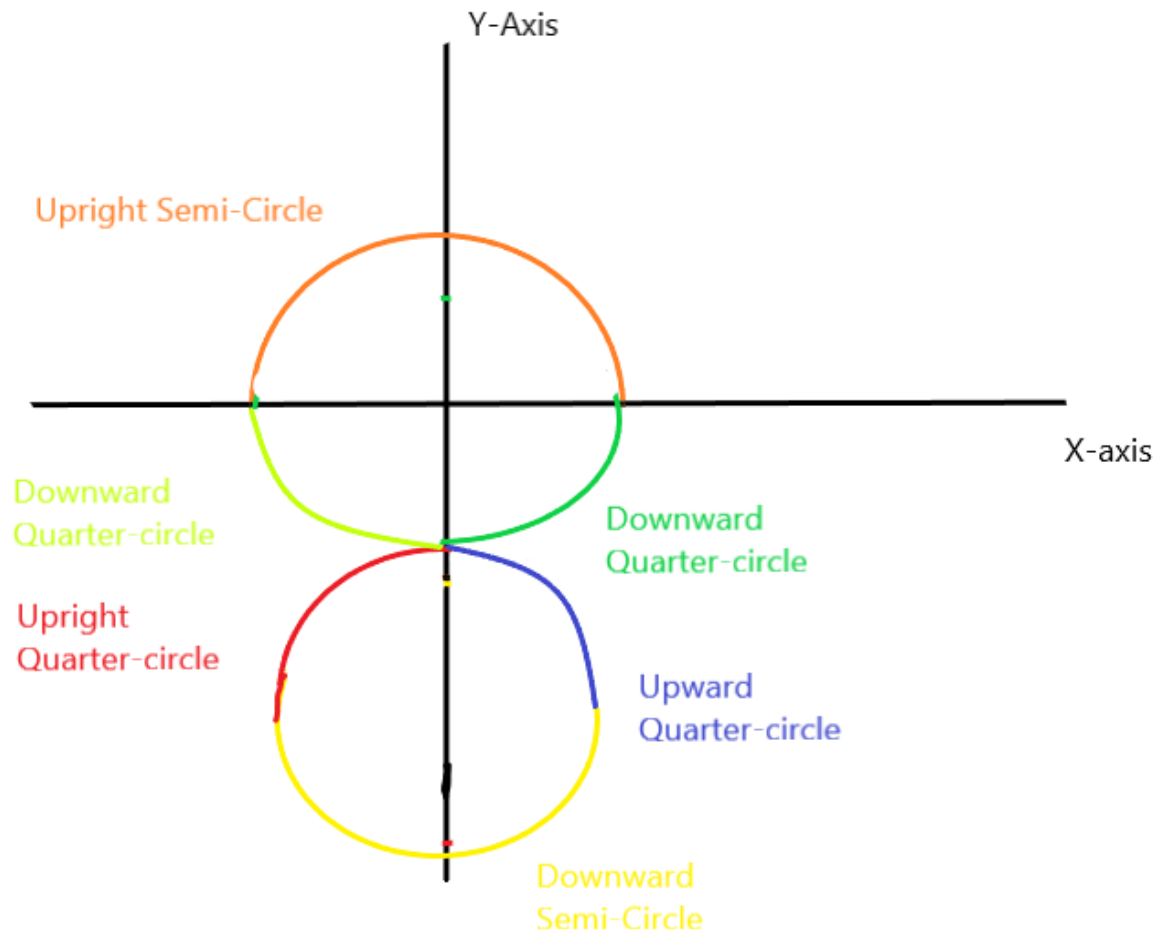
$$\text{New Longitude} = \text{Home Longitude} + x * \frac{180}{\pi * \text{Earth Radius} * \cos(\text{Home Longitude} * \pi / 180)}$$

$$\text{New Latitude} = \text{Home Latitude} + y * \frac{180}{\pi * \text{Earth Radius}}$$

3. In the function `adds_hexagon_mission()`, calling the `get_hexagon_points()` and then adding those latitude and longitudes into waypoint commands . And creating the mission.

EIGHT MISSION

1. The function `get_eight_points()` solves the equation of circle $x^2 + y^2 = r^2$ and saves it in `coords` list.



First, it creates an upright semi-circle and saves the coordinates in `coords1` taking origin (0, 0) as centre (in latitude and longitude, it is the home location)

Second, from previous end point it creates a downward quarter-circle and saves the coordinates in `coords2`.

Third, from previous end point it creates an upright quarter-circle taking (0, 2*RADIUS) (because of error in calculation, the centre

coordinate is (0, 2.32*RADIUS) instead) as centre and saves the coordinates in coords3.

Fourth, from previous end point it creates a downward semi-circle taking the new centre and saves the coordinates in coords4.

Fifth, from previous end point it creates an upright quarter circle and saves the coordinates in coords5.

Sixth, from previous end point it creates a downward quarter circle taking (0,0) as centre and saving the coordinates in coords6.

2. Now converting those cartesian coordinates (x, y) into (latitude, longitude) by using the below transformation.

$$New\ Longitude = Home\ Longitude + x * \frac{180}{\pi * Earth\ Radius * \cos(Home\ Longitude * \pi / 180)}$$

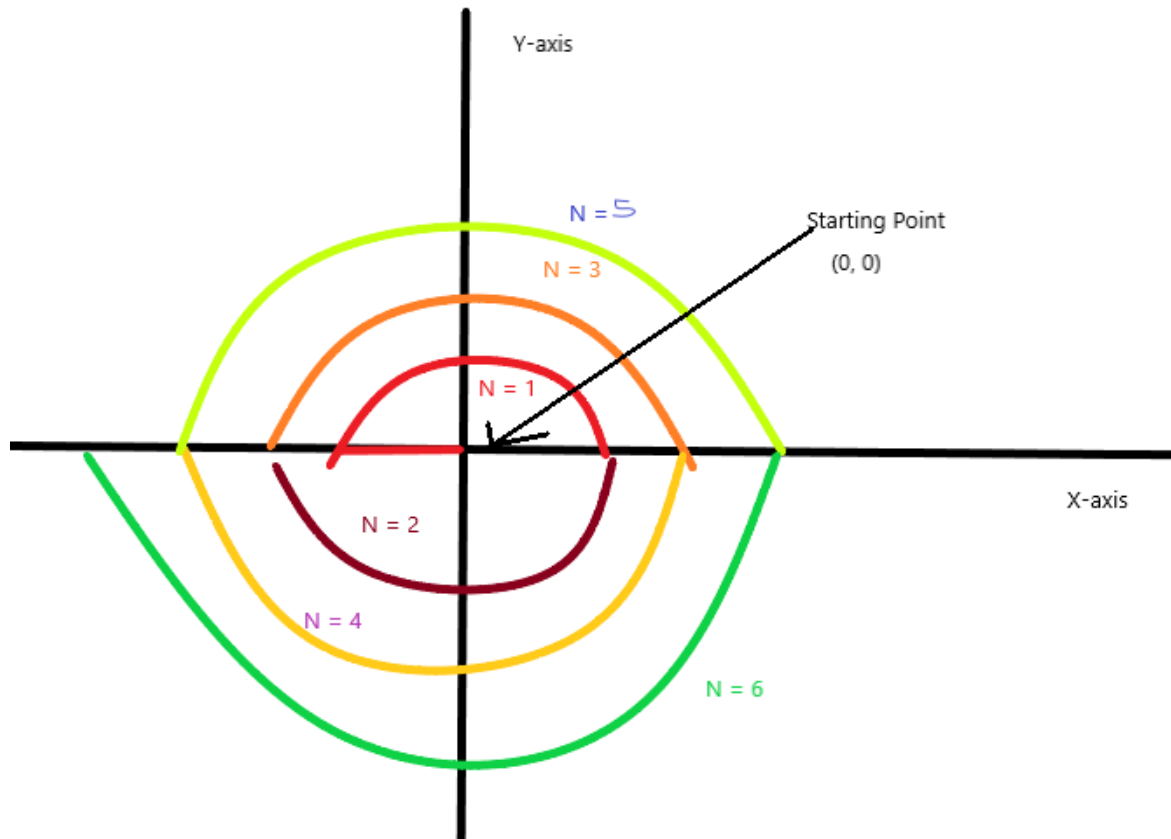
$$New\ Latitude = Home\ Latitude + y * \frac{180}{\pi * Earth\ Radius}$$

And returning them.

3. In the function adds_eight_mission(), calling the get_eight_points() and then adding those latitude and longitudes into waypoint commands. And creating the mission.

SPIRAL MISSION

1. In the function, `get_spiral_points()`, solves the equation of circle $x^2 + y^2 = r^2$ and saves it in `coords` list.
Taking the starting point as origin.



All upright semi-circles have centre at origin (0, 0) and all downward semi-circles have centre at (-SMALLEST_SEMICIRCLE_RADIUS, 0)

For every semi-circle in the mission, the radius is $N \times \text{SMALLEST_SEMICIRCLE_RADIUS}$, where N is the number of semi-circle, starting from smallest semi-circle as $N = 1$.

2. Now converting those cartesian coordinates (x, y) into (latitude, longitude), and returning them.
3. In the function `adds_spiral_mission()`, calling the `get_spiral_points()` and then adding those latitude and longitudes into waypoint commands. And creating the mission.

VERTICAL EIGHT MISSION

1. The function, `get_vertseight_points()` does the the same job as `get_eight_points()` but in vertical condition I have taken latitude same and varied longitude accordingly with the altitude.

$$\text{New Longitude} = \text{Home Longitude} + x * \frac{180}{\pi * \text{Earth Radius} * \cos(\text{Home Longitude} * \pi / 180)}$$

$$\text{New Latitude} = \text{Home Latitude}$$

$$\text{New Altitude} = y + \text{Home altitude}$$

Now transforming cartesian coordinate **x** into **longitude** using above transformation.

This time the **y** coordinates are not transformed into latitude rather they are summed up to the takeoff/home altitude and then returned.

(Taking appropriate home altitude such that drone/quad dose not hit ground when doing mission)

2. The function `adds_verteight_mission()` calls the `get_verteight_points()` and then adding latitude and longitudes this time altitudes also into waypoint commands. And creating the mission.

VERTICAL CIRCLE MISSION

1. The function, `get_vertcircle_points()` solves the equation of circle $x^2 + y^2 = r^2$ and saves it in `coords` list.

$$\text{New Longitude} = \text{Home Longitude} + x * \frac{180}{\pi * \text{Earth Radius} * \cos(\text{Home Longitude} * \pi / 180)}$$

$$\text{New Latitude} = \text{Home Latitude}$$

$$\text{New Altitude} = y + \text{Home altitude}$$

but in vertical condition I have taken latitude same and varied longitude accordingly with the altitude.

Now converting cartesian coordinate `x` into `longitude`, using above transformation.

This time the `y` coordinates are not transformed rather they are summed up with the takeoff/home altitude and then returned.

(Taking appropriate home altitude such that drone/quad dose not hit ground when doing mission)

2. The function `adds_vertcircle_mission()` calls the `get_vertcircle_points()` and then adding latitude and longitudes this time altitudes also into waypoint commands. And creating the mission.

FLIP

1. The function, `start_flip()` I am overriding channels using `Vehicle.channels.overrides` function. I am first arming the drone in “Stabilize” flight mode and overriding the channel 3 or throttle for takeoff, as you would have done in flying your quad by using remote control. Then I am changing flight mode to “Flip” that obviously, does a flip 😊.